Lecture 4
Advanced SQL for Query

Dr. Yu Yang
Department of Mathematics and Information Technology
The Education University of Hong Kong

All teaching and learning materials of INT6164 MUST NOT be distributed or shared without the permission of the Instructor!

Outline

- SQL Major Aspects ✓
- Basic SQL Queries
- Set Operation
- Aggregate Functions & Group By, Having and Order By Clauses
- Nested Queries
- NULL Values
- Text-to-SQL & Natural Language to SQL

SQL Major Aspects

- A major strength of the relational model is that it supports simple and powerful ***querying*** of data
- Structured Query Language (SQL) is the most widely used commercial relational database language
- SQL has several aspects to it:
  1. Data Manipulation Language (DML)
     - It allows users to pose queries and insert, delete and modify rows
  2. Data Definition Language (DDL)
     - It allows users to create, delete, and modify tables and views

SQL Major Aspects

- SQL has several aspects to it:
  3. Triggers and Advanced Integrity Constraints
     - It supports “triggers”, which are actions executed by the DBMS whenever changes to the database meet conditions specified in triggers
  4. Embedded and Dynamic Language
     - Embedded SQL allows SQL code to be called from a *host language* (e.g., Java)
     - Dynamic SQL allows SQL queries to be constructed and executed at run-time

SQL Major Aspects

- SQL has several aspects to it:
  5. Remote Database Access
     - It allows connecting client programs to remote database servers
  6. Transaction Management
     - It allows users to explicitly control aspects of how a transaction is to be executed
  7. Security
     - It provides mechanisms to control users' accesses to data objects (e.g., tables and views)

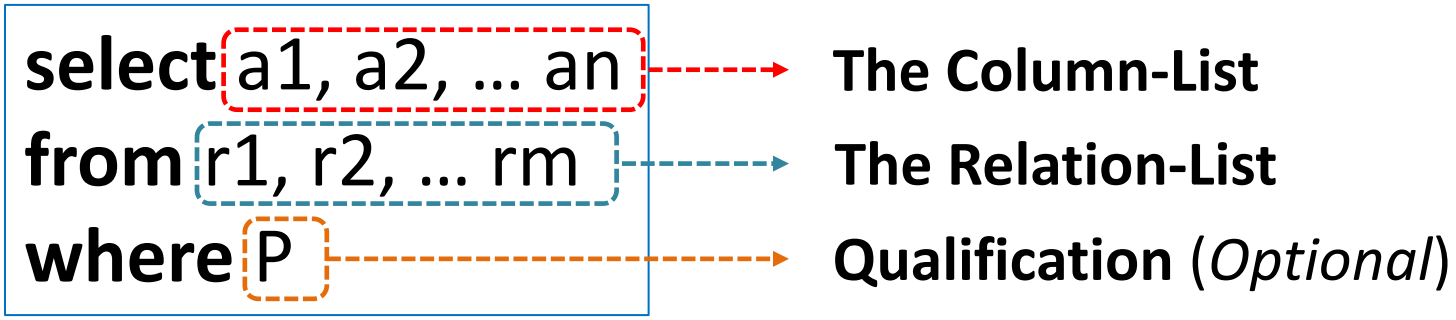
And others…

Outline

- SQL Major Aspects
- Basic SQL Queries ✓
- Set Operation
- Aggregate Functions & Group By, Having and Order By Clauses
- Nested Queries
- NULL Values
- Text-to-SQL & Natural Language to SQL

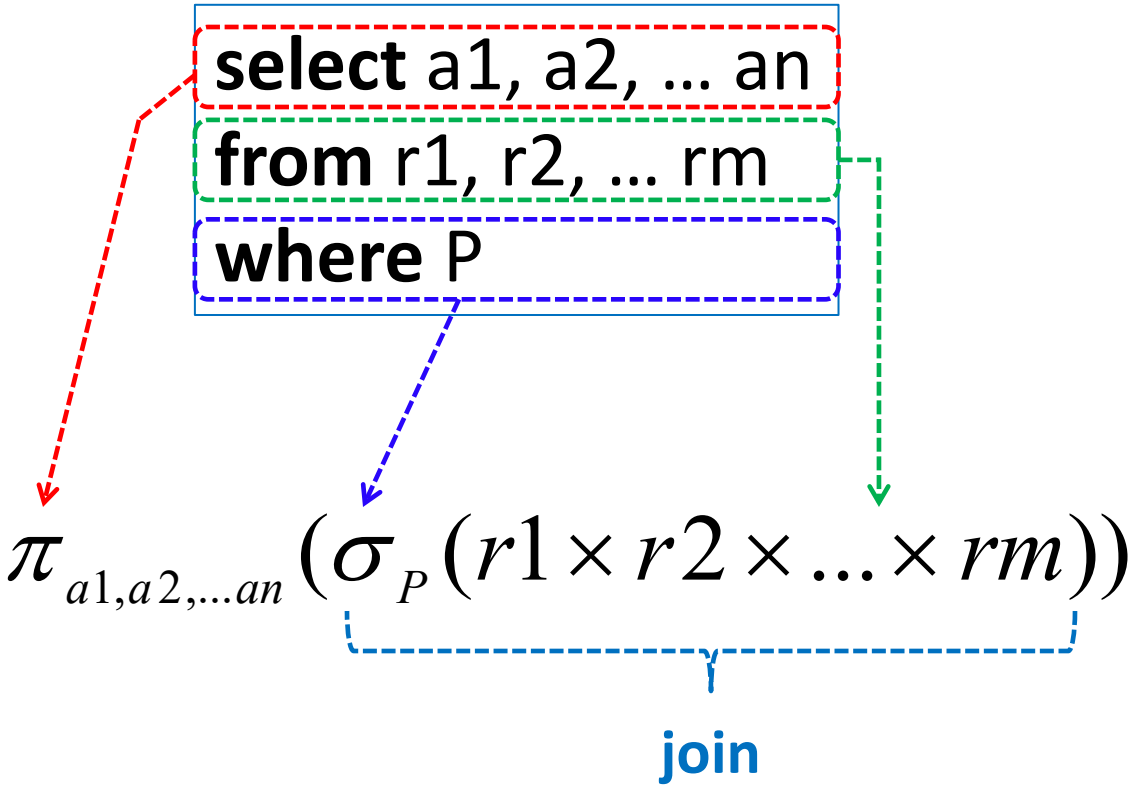
Basic SQL Queries

- The basic form of an SQL query is as follows:



Equivalence to Relational Algebra

- The basic form of an SQL query is as follows:



Reminder: Our Mini-U DB

STUDENT		
Ssn	Name	Address
123	smith	main str
234	jones	QF ave

CLASS		
c-id	c-name	units
15-415	s.e.	2
15-413	o.s.	2

SSN	**c-id**	grade
123	15-415	A
234	15-413	B

The WHERE Clause

- Find the ssn(s) of everybody called “smith”

STUDENT		
Ssn	Name	Address
123	smith	main str
234	jones	QF ave

```
select    ssn
from      student
where     name='smith'
```

The WHERE Clause

- Find ssn(s) of all “smith”s on “main str”

STUDENT		
Ssn	**Name**	**Address**
123	smith	main str
234	jones	QF ave

```
select   ssn
from     student
where    address='main' and name = 'smith'
```

The WHERE Clause

- Boolean operators (**and, or, not**)
- Comparison operators (<, ≤, >, ≥, =, ≠)
- And more…

What About Strings?

- Find student ssn(s) who live on “main” (st or str or street – i.e., “main st” or “main str” or “main street”)

```
select    ssn
from      student
where     address like ‘main%’
```

%: Variable-length do not care (i.e., stands for 0 or more arbitrary characters)

Another Example on *Pattern Matching*

- Find the ages of sailors whose names begin and end with B and have at least 3 characters

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

```
select   S.age
from     Sailors S
where    S.sname like 'B_%B'
```

_: Single-character do not care (i.e., stands for any 1 character)

%: Wildcard 通配符. %B: A string ending with the letter B;
B%: A string starting with the letter B; %B%: A string containing the letter B.

The FROM Clause

- Find the names of students taking 15-415

STUDENT		
Ssn	Name	Address
123	smith	main str
234	jones	QF ave

CLASS		
c-id	c-name	units
15-415	s.e.	2
15-413	o.s.	2



2-way Join!

SSN	c-id	grade
123	15-415	A
234	15-413	B

```
select  Name
from    STUDENT, TAKES
where   ???
```

The FROM Clause

- Find the names of students taking 15-415

```
select      Name
from        STUDENT, TAKES
where       STUDENT.ssn = TAKES.ssn
                  and TAKES.c-id = '15-415'
```

Renaming: Tuple Variables

- Find the names of students taking 15-415

```
select    Name
from      STUDENT as S, TAKES as T
where     S.ssn = T.ssn
                  and T.c-id = '15-415'
```

Optional!

Renaming: Self-Joins

- Find Tom's grandparent(s)

PC	
p-id	c-id
Mary	Tom
Peter	Mary
John	Tom

PC	
p-id	c-id
Mary	Tom
Peter	Mary
John	Tom

```
select    gp.p-id
from      PC as gp, PC
where     gp.c-id = PC.p-id
            and  PC.c-id = 'Tom'
```

More on Self-Joins

- Find names and increments for the ratings of persons who have sailed two different boats on the same day

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

```
select   S.sname, S.rating+1 as rating
from     Sailors S, Reserves R1, Reserves R2
where    S.sid = R1.sid and S.sid = R2.sid and
         R1.day = R2.day and R1.bid != R2.bid
```

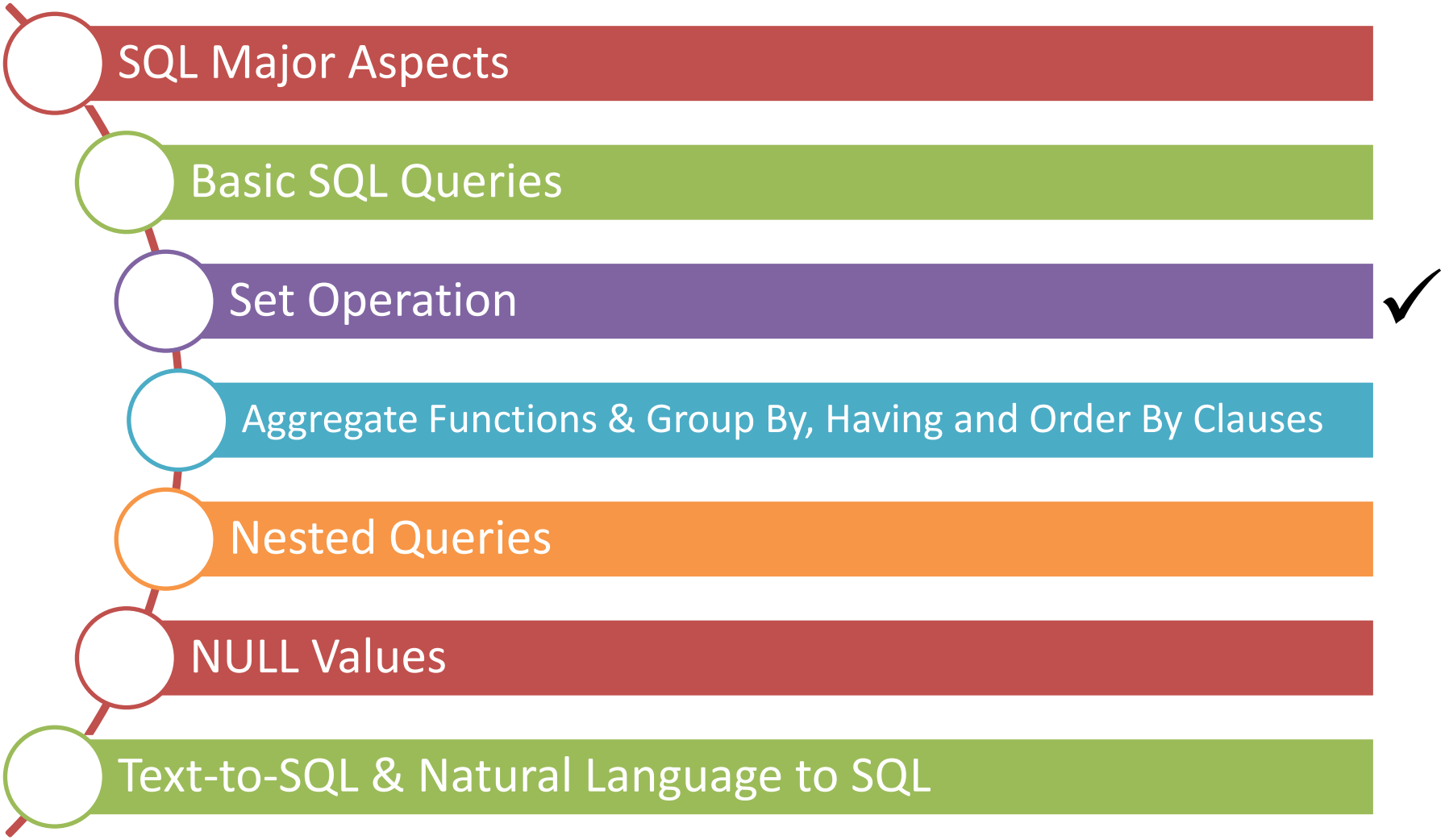
Renaming: Theta Joins

- Find course names with more units than 15-412

CLASS		
c-id	**c-name**	**units**
15-413	s.e.	2
15-412	o.s.	2

```
select   c1.c-name
from     class as c1, class as c2
where    c1.units > c2.units
         and  c2.c-id = '15-412'
```

Outline


SQL Major Aspects
Basic SQL Queries
Set Operation
Aggregate Functions & Group By, Having and Order By Clauses
Nested Queries
NULL Values
Text-to-SQL & Natural Language to SQL

Set Operations

- Find ssn(s) of students taking both 15-415 and 15-413

TAKES		
SSN	**c-id**	**grade**
123	15-415	A
234	15-413	B

```
select  ssn
from    takes
where   c-id='15-415' and c-id='15-413'
```

Set Operations

- Find ssn(s) of students taking both 15-415 and 15-413

TAKES		
SSN	c-id	grade
123	15-415	A
234	15-413	B

(select ssn **from** takes **where** c-id="15-415")
intersect
(select ssn **from** takes **where** c-id="15-413")

Other operations: **union** , **except**

Set Operations

- Find ssn(s) of students taking both 15-415 or 15-413

TAKES		
SSN	c-id	grade
123	15-415	A
234	15-413	B

(select ssn **from** takes **where** c-id="15-415")
union
(select ssn **from** takes **where** c-id="15-413")

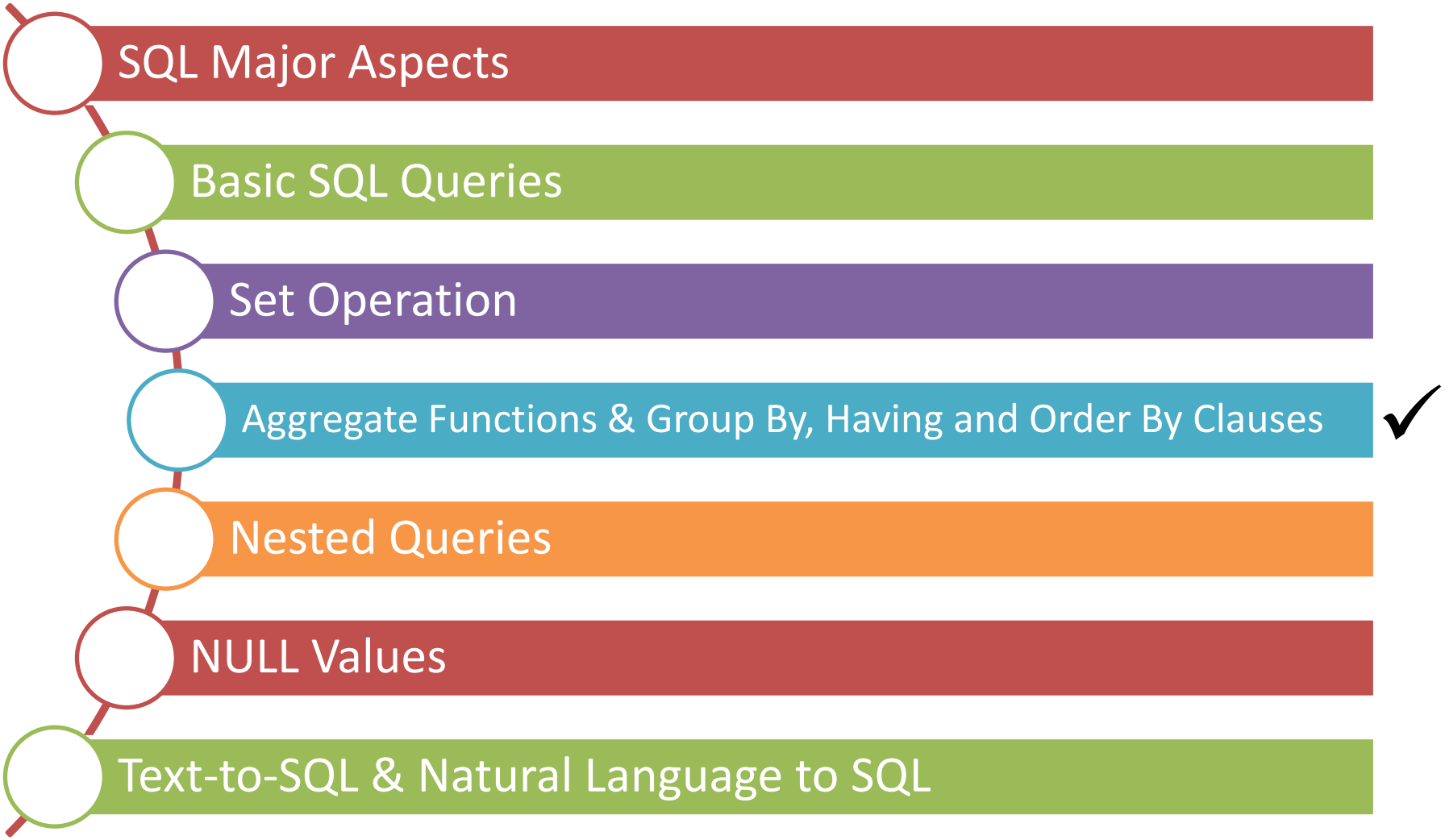
Set Operations

- Find ssn(s) of students taking both 15-415 but not 15-413

TAKES		
SSN	c-id	grade
123	15-415	A
234	15-413	B

(select ssn **from** takes **where** c-id="15-415")
except
(select ssn **from** takes **where** c-id="15-413")

Outline


SQL Major Aspects
Basic SQL Queries
Set Operation
Aggregate Functions & Group By, Having and Order By Clauses
Nested Queries
NULL Values
Text-to-SQL & Natural Language to SQL

Aggregate Functions

- Find average grade, across all students

TAKES		
SSN	c-id	grade
123	15-413	4
234	15-413	3

```
select ??
from takes
```

Aggregate Functions

- Find average grade, across all students

TAKES		
SSN	**c-id**	**grade**
123	15-413	4
234	15-413	3

```
select    avg(grade)
from      takes
```

Other functions: Count ([Distinct] A), Sum ([Distinct] A), Max (A), Min (A), assuming column A

Aggregate Functions

- Find total number of enrollments

TAKES		
SSN	c-id	grade
123	15-413	4
234	15-413	3

```
select  count(*)
from    takes
```

Aggregate Functions

- Find total number of students in 15-415

TAKES		
SSN	c-id	grade
123	15-413	4
234	15-413	3

```
select   count(*)
from     takes
where    c-id='15-415'
```

Aggregate Functions

- Find the name and age of the oldest sailor

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

```
select S.sname, max(S.age)
from Sailors S
```

This query is illegal in SQL
If the "select" clause uses an aggregate function, it must use **ONLY** aggregate function unless the query contains a "group by" clause!

GROUP BY Clauses

- Find the age of the youngest sailor for each rating level

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

  - In general, we do not know how many rating levels exist, and what the rating values for these levels are!

  - Suppose we know that rating values go from 1 to 10; we can write 10 queries that look like this (!):

For $i = 1, 2, \ldots, 10$:

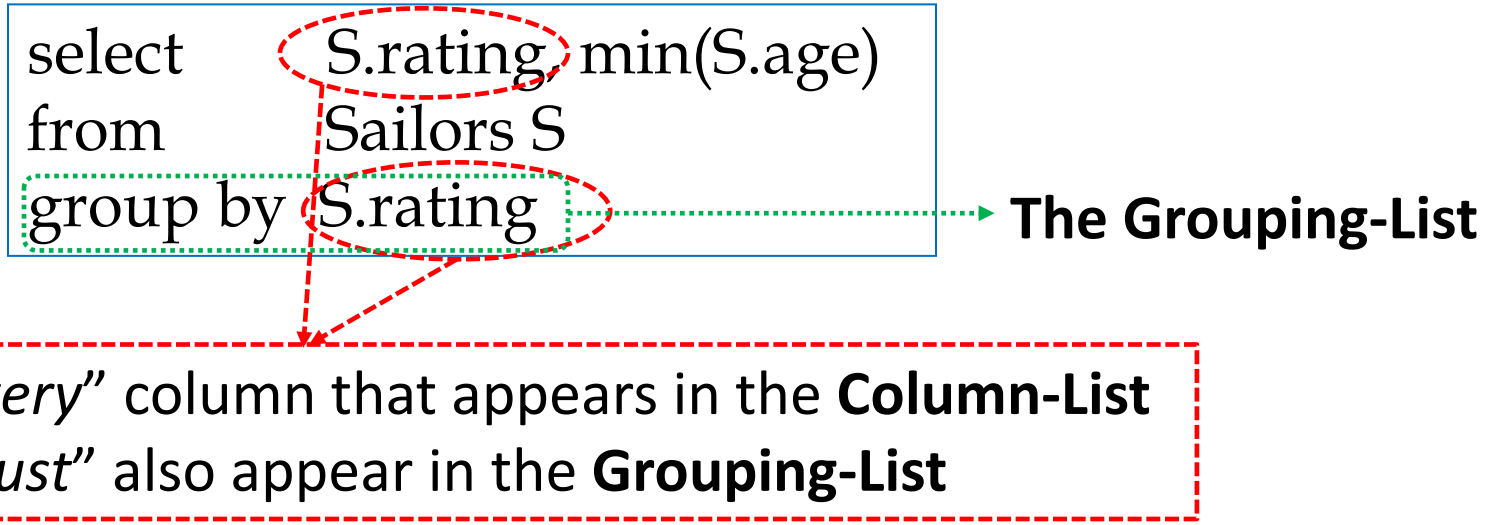
```
SELECT   min(S.age)
FROM     Sailors S
WHERE    S.rating = i
```

GROUP BY Clauses

- Find the age of the youngest sailor for each rating level

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

  - Using the GROUP BY clause, we can write this query as follows:



HAVING Clauses

- Find age of the youngest sailor with age ≥ 18, for each rating level with at least 2 sailors

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

```
SELECT      S.rating, min(S.age)
FROM        Sailors S
WHERE       S.age >= 18
GROUP BY    S.rating
HAVING      COUNT (*) > 1
```

HAVING Clauses

- Find age of the youngest sailor with age ≥ 18, for each rating level with at least 2 sailors

rating	age
7	45.0
1	33.0
8	55.5
8	25.5
10	35.0
7	35.0
10	16.0
9	35.0
3	25.5
3	63.5
3	25.5



rating	age
1	33.0
3	25.5
3	63.5
3	25.5
7	45.0
7	35.0
8	55.5
8	25.5
9	35.0
10	35.0



rating	minage
3	25.5
7	35.0
8	25.5

```
SELECT    S.rating, min(S.age)
FROM      Sailors S
WHERE     S.age >= 18
GROUP BY  S.rating
HAVING    COUNT (*) > 1
```

ORDER BY Clause

- Find student records, sorted in name order

```
select *
from student
where ??
```

ORDER BY Clause

- Find student records, sorted in name order

```
select *
from student
order by name asc
```

ascending order is the default

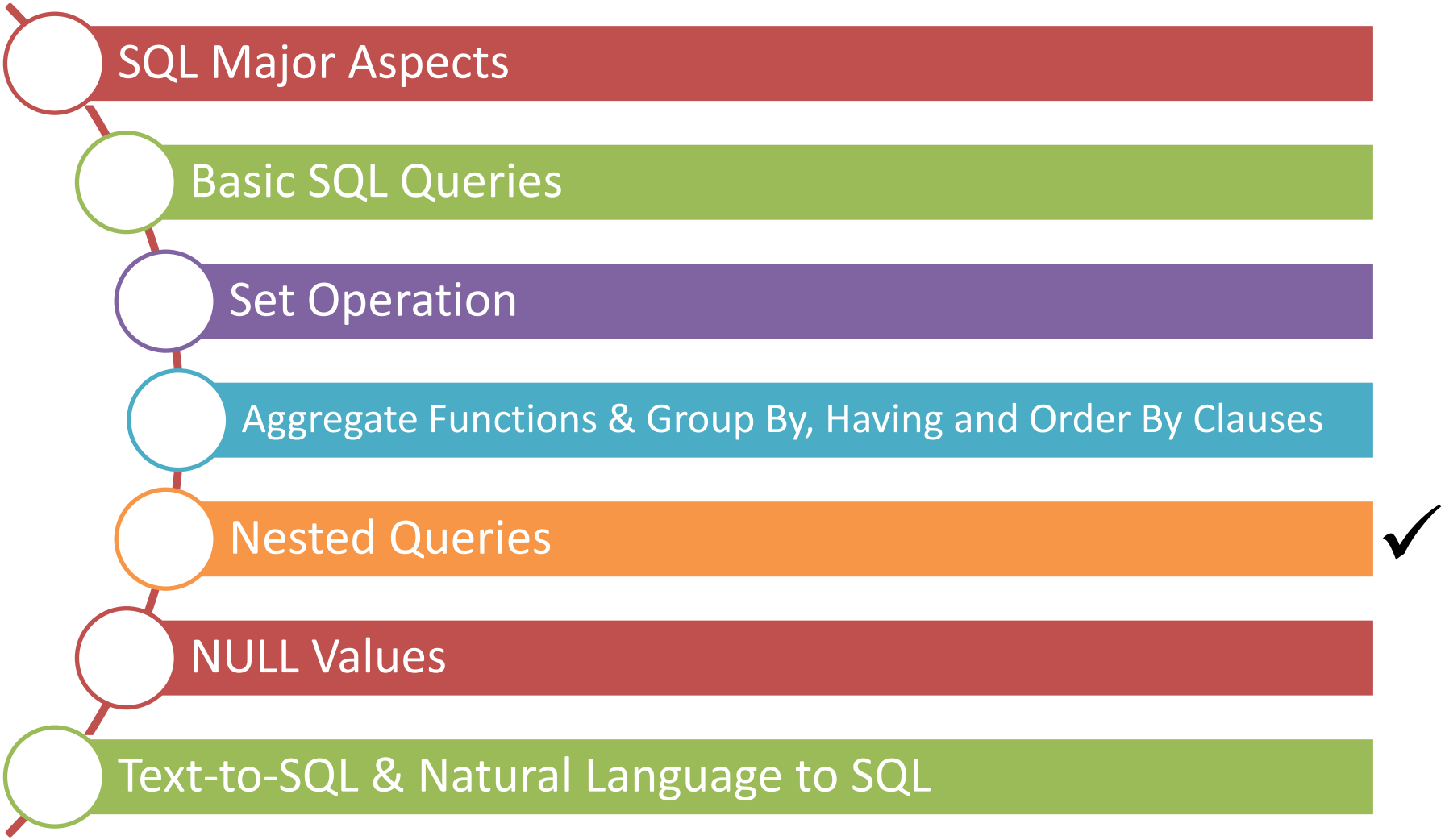
Can use **desc** to sort in descending order

ORDER BY Clause

- Find student records, sorted in name order; break ties by reverse ssn

```
select *
from student
order by name, ssn desc
```

Outline


SQL Major Aspects
Basic SQL Queries
Set Operation
Aggregate Functions & Group By, Having and Order By Clauses
Nested Queries
NULL Values
Text-to-SQL & Natural Language to SQL

A Join Query

- Find the names of sailors who have reserved boat 101

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

```
select  S.sname
from    Sailors S, Reserves R
where   S.sid = R.sid and R.bid = 101
```

Nested Queries

- Find the names of sailors who have reserved boat 101

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

OR...

```
SELECT  S.sname
FROM    Sailors S
WHERE   S.sid IN (SELECT  R.sid
                  FROM    Reserves R
                  WHERE   R.bid=101)
```

IN compares a value with a set of values

Nested Queries

- Find the names of sailors who have not reserved boat 101

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

```
SELECT   S.sname
FROM     Sailors S
WHERE    S.sid NOT IN (SELECT R.sid
                       FROM   Reserves R
                       WHERE  R.bid=101)
```

Deeply Nested Queries

- Find the names of sailors who have reserved a red boat

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

Boats		
Bid	**Bname**	**Color**
101	Interlake	Red
102	Clipper	Green

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid IN  (SELECT  R.sid
                  FROM  Reserves R
                  WHERE  R.bid IN (SELECT B.bid
                                   FROM Boats B
                                   WHERE B.color = 'red'))
```

In principle, queries with very deeply nested structures are possible!

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid IN  (SELECT  R.sid
                  FROM  Reserves R
                  WHERE  R.bid IN (SELECT B.bid
                                   FROM Boats B
                                   WHERE B.color = 'red'))
```

Sailors instance:

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Reserves instance:

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats instance:

Bid	Bname	Color
101	Interlake	Blue
102	Interlake	Red
103	Clipper	Green
104	Marine	Red

Deeply Nested Queries

- Find the names of sailors who have not reserved a red boat

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

Boats		
Bid	**Bname**	**Color**
101	Interlake	Red
102	Clipper	Green

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid NOT IN  (SELECT  R.sid
                      FROM  Reserves R
                      WHERE  R.bid IN (SELECT B.bid
                                       FROM Boats B
                                       WHERE B.color = 'red'))
```

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid NOT IN (SELECT  R.sid
                     FROM  Reserves R
                     WHERE  R.bid IN (SELECT B.bid
                                      FROM Boats B
                                      WHERE B.color = 'red'))
```

Sailors instance:

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Reserves instance:

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats instance:

Bid	Bname	Color
101	Interlake	Blue
102	Interlake	Red
103	Clipper	Green
104	Marine	Red

This returns the names of sailors who have not reserved a red boat!

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid IN (SELECT  R.sid
                 FROM  Reserves R
                 WHERE  R.bid NOT IN (SELECT B.bid
                                      FROM Boats B
                                      WHERE B.color = 'red'))
```

Sailors instance:

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Reserves instance:

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats instance:

Bid	Bname	Color
101	Interlake	Blue
102	Interlake	Red
103	Clipper	Green
104	Marine	Red

This returns the names of sailors who have reserved a boat that is not red.

≠

The previous one returns the names of sailors who have not reserved a red boat!

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.sid NOT IN (SELECT  R.sid
                     FROM  Reserves R
                     WHERE  R.bid NOT IN (SELECT B.bid
                                          FROM Boats B
                                          WHERE B.color = 'red'))
```

Sailors instance:

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Reserves instance:

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Boats instance:

Bid	Bname	Color
101	Interlake	Blue
102	Interlake	Red
103	Clipper	Green
104	Marine	Red

This returns the names of sailors who have not reserved a boat that is not red!

=

As such, it returns names of sailors who have reserved only red boats (*if any*)

Correlated Nested Queries

- Find the names of sailors who have reserved boat 101

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

Compares a value with a set of values

```
SELECT S.sname
FROM Sailors S
WHERE S.sid IN (SELECT R.sid
                FROM Reserves R
                WHERE R.bid=101)
```

Allows us to test whether a set is "nonempty"

```
SELECT S.sname
FROM Sailors S
WHERE EXISTS (SELECT *
              FROM Reserves R
              WHERE R.bid=101
              AND R.sid = S.sid)
```

A correlation

Correlated Nested Queries

- Find the names of sailors who have not reserved boat 101

Sailors			
Sid	**Sname**	**Rating**	**age**
22	Dustin	7	45.0
29	Brutus	1	33.0

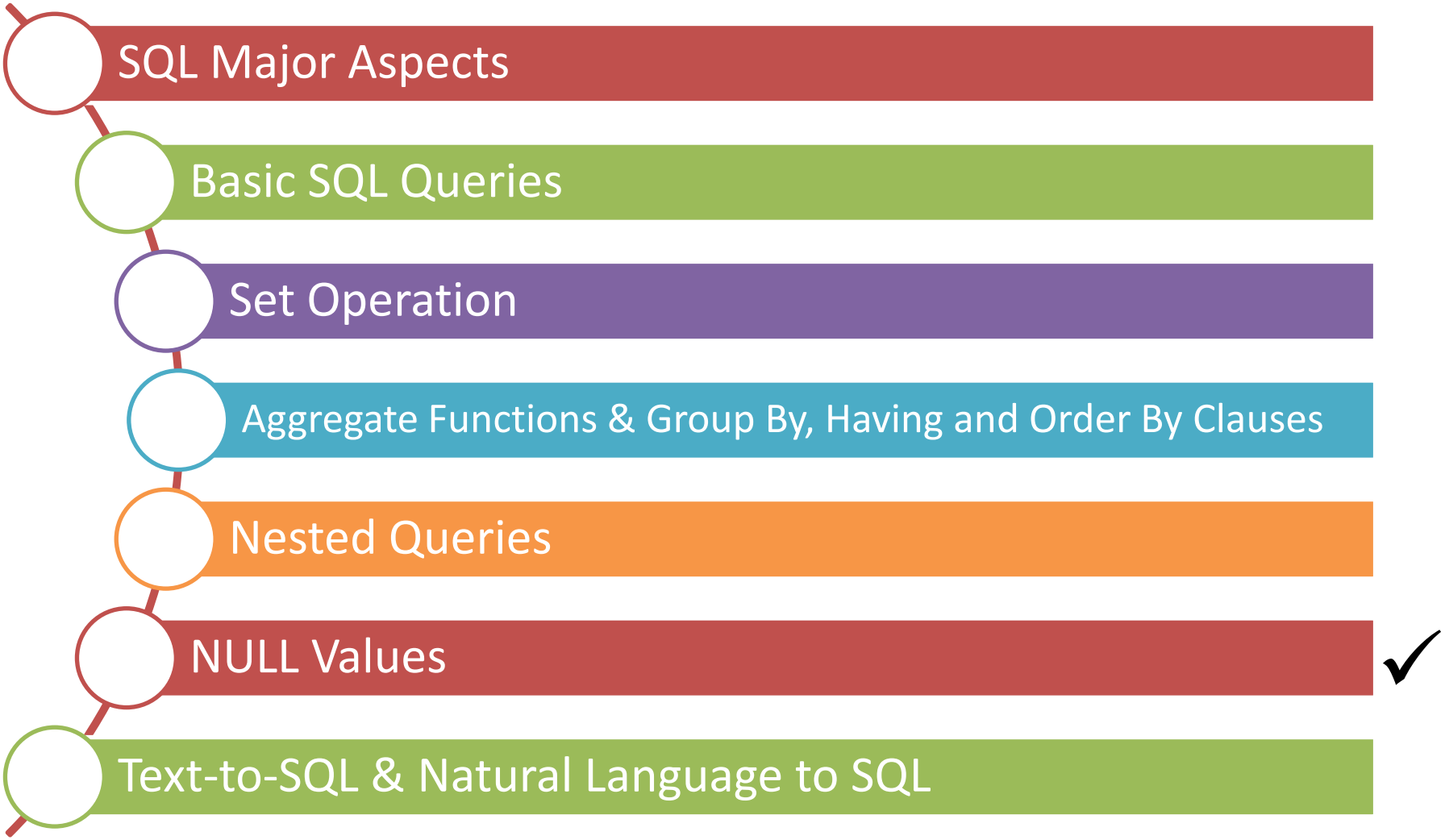
Reserves		
Sid	**Bid**	**Day**
22	101	10/10/2013
22	102	10/10/2013

```
SELECT S.sname
FROM Sailors S
WHERE S.sid NOT IN (SELECT R.sid
                    FROM Reserves R
                    WHERE R.bid=101)
```

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (SELECT *
                  FROM Reserves R
                  WHERE R.bid=101
                  AND R.sid = S.sid)
```



Outline


SQL Major Aspects
Basic SQL Queries
Set Operation
Aggregate Functions & Group By, Having and Order By Clauses
Nested Queries
NULL Values
Text-to-SQL & Natural Language to SQL

NULL Values

- Column values can be *unknown* (e.g., a sailor may not yet have a rating assigned)
- Column values may be *inapplicable* (e.g., a maiden-name column for men!)
- The **NULL** value can be used in such situations
- However, the NULL value complicates many issues!
  - Using NULL with aggregate operations
    - COUNT (*) handles NULL values like any other values
    - SUM, AVG, MIN, and MAX discard NULL values
  - Comparing NULL values to valid values
  - Comparing NULL values to NULL values

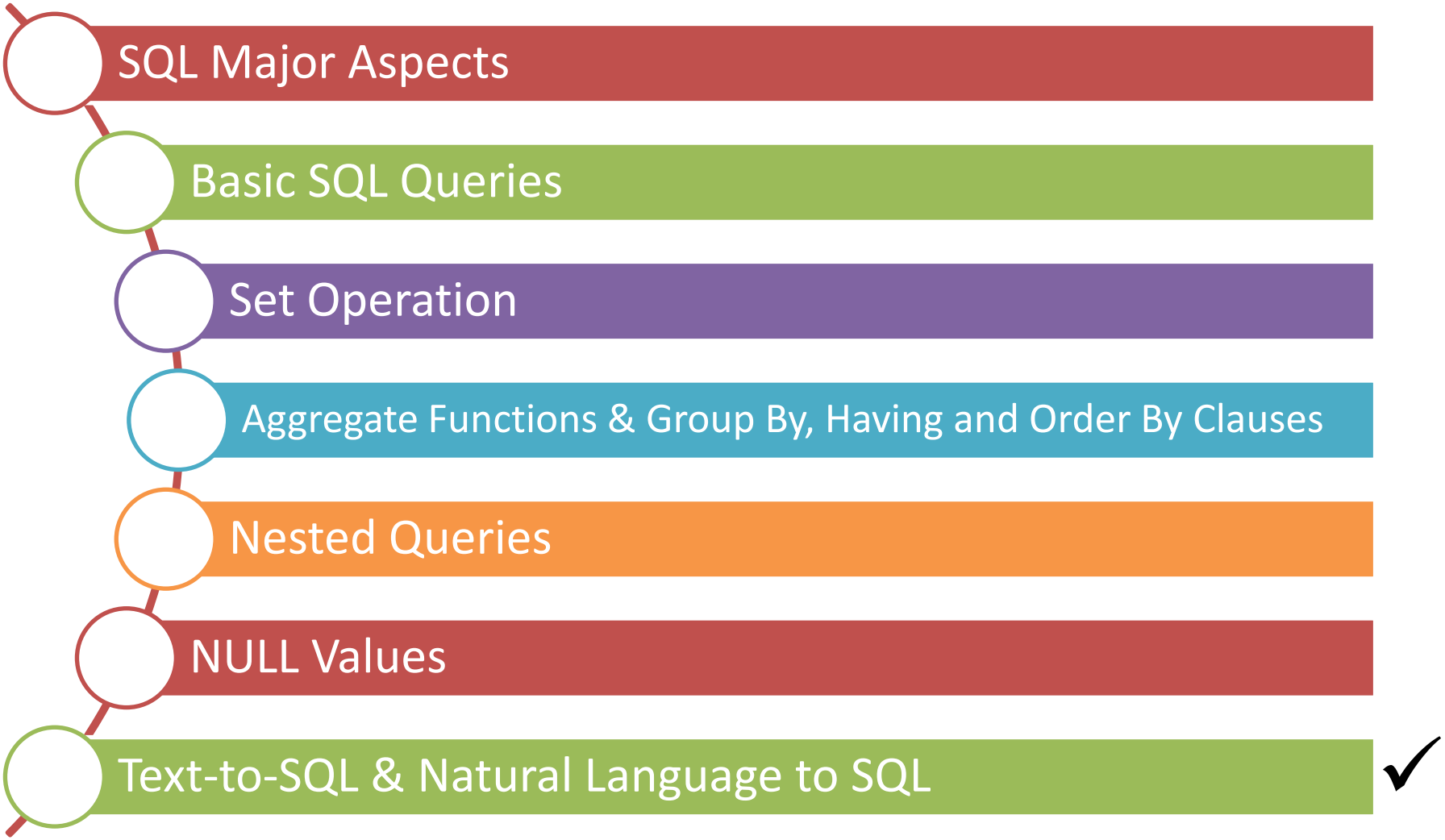
Comparing Values In the Presence of NULL

- Considering a row with rating = NULL and age = 20; what will be the result of comparing it with the following rows?
  - Rating = 8 OR age < 40 → TRUE
  - Rating = 8 AND age < 40 → unknown

- In general:
  - NOT unknown → unknown
  - True OR unknown → True
  - False OR unknown → unknown
  - False AND unknown → False
  - True AND unknown → unknown
  - Unknown [AND|OR] unknown → unknown

In the context of *duplicates*, the comparison of two NULL values is implicitly treated as TRUE (Anomaly!)

Outline


SQL Major Aspects
Basic SQL Queries
Set Operation
Aggregate Functions & Group By, Having and Order By Clauses
Nested Queries
NULL Values
Text-to-SQL & Natural Language to SQL

Bridging Basic SQL to Automation

Human SQL Writing

Rigid Syntax: Requires exact keyword order (SELECT, FROM, WHERE, GROUP BY).

Manual Join Traversal: Developer manually tracks Primary / Foreign Key links.

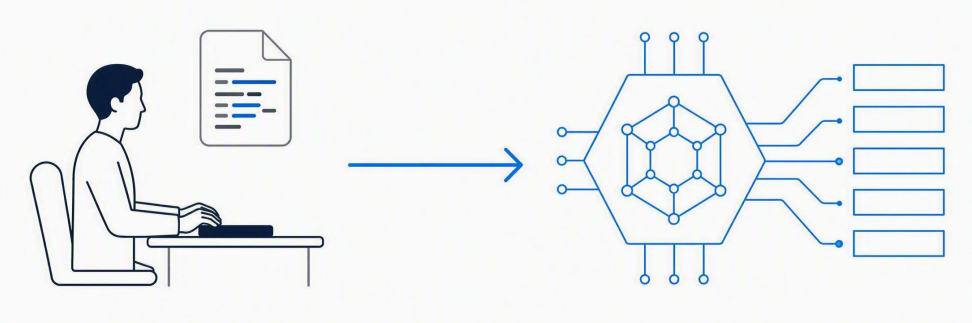
Explicit Schema Knowledge: Developer must memorize exact table and column names.

Automated Generation

Intent Parsing: Maps fuzzy human concepts ("top earners") to numeric SQL bounds.

Automated Schema Linking: Dynamically resolves relationship graphs between entities.

Self-Correction: Uses database execution logs to debug syntax errors automatically.



Defining Text-to-SQL vs. NL-to-SQL

Text-to-SQL (The Core Translation Engine)

Definition: The algorithmic process of converting a structured or semi-structured natural language prompt into a syntactically and semantically valid SQL query given a fixed database schema context.

Scope: Technical model-level task (Prompt + Schema $\rightarrow$ SQL Statement). Single-pass or modular execution.

Natural Language to SQL (NL-to-SQL - The End-to-End System)

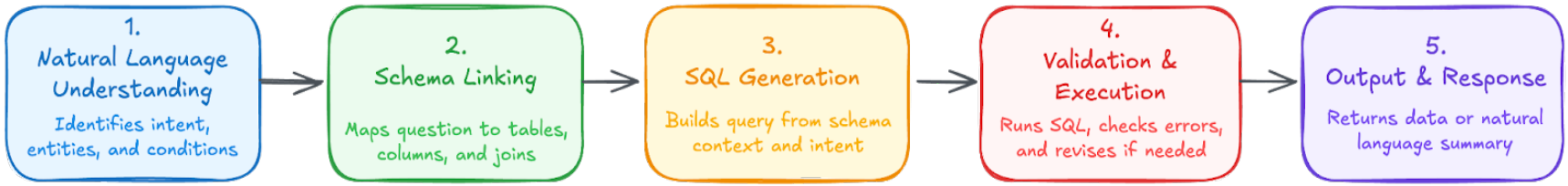
Definition: The holistic user-facing solution that manages natural human conversation, clarifies ambiguous business jargon, retrieves relevant metadata, generates and executes SQL, and formats the output.

Scope: Enterprise application-level system (User Query $\rightarrow$ Clarification $\rightarrow$ Schema Retrieval $\rightarrow$ Text-to-SQL $\rightarrow$ DB Execution $\rightarrow$ Visual Chart / Summary).

Text-to-SQL vs. NL-to-SQL

Dimension	Text-to-SQL	NL-to-SQL System
Primary Focus	Code translation & SQL syntax accuracy	End-to-end user query resolution
Input Nature	Clean, direct text prompts with schema	Fuzzy, conversational multi-turn dialogue
Schema Handling	Pre-filtered DDLs (data definition languages) passed directly in prompt	Dynamic RAG / Ontology graph retrieval
Ambiguity Resolution	Makes best-guess statistical assumption	Asks user clarifying questions dynamically
Execution & Recovery	Outputs raw SQL string (no execution)	Executes SQL, self-corrects on DB errors
Final Output	SQL String (`SELECT ...`)	Tabular data, charts, natural language answer

5-Stage Processing Pipeline



1. **Stage 1: Intent Understanding & Entity Parsing**
   Extract requested metrics, filter conditions, groupers, and sorting requirements.

2. **Stage 2: Schema Linking & Context Retrieval**
   Map extracted entities to exact database tables, column names, and JOIN foreign keys.

3. **Stage 3: AST / Intermediate Representation & SQL Generation**
   Construct Abstract Syntax Tree, order clauses, and generate target SQL syntax.

4. **Stage 4: Execution & Agentic Self-Correction Loop**
   Run SQL against database engine. If an error occurs, feed error log back to LLM to fix query.

5. **Stage 5: Result Formatting & Natural Language Explanation**
   Convert raw database tuples into natural answers, tables, or visual charts.

https://www.puppygraph.com/blog/text-to-sql-llm

Stage 1 & Stage 2: Intent & Schema Linking

Stage 1: Intent Decomposition

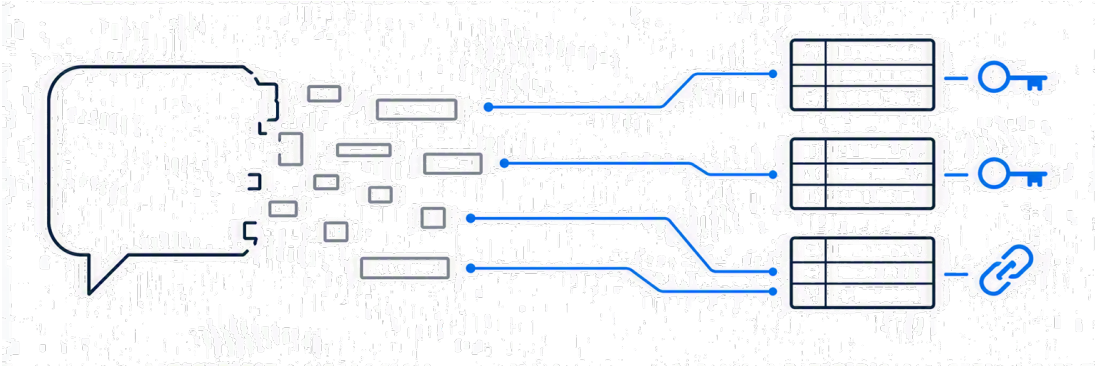
Deconstructs raw text into fundamental SQL building blocks:

- **Target Projection:** What columns to display (`SELECT`).
- **Filtering Logic:** Value constraints (`WHERE / HAVING`).
- **Aggregations:** Sums, counts, averages (`SUM, COUNT`).
- **Grouping & Order:** Attributes to aggregate by (`GROUP BY, ORDER BY`).

Stage 2: Schema Linking

Bridges human vocabulary with physical database schema:

- **Synonym Resolution:** "Client" → `customers` table.
- **Foreign Key Pathing:** Finds valid join paths between separated tables.
- **Metadata Injection:** Injecting column descriptions and sample values into prompt context.



Stage 3 & Stage 4: Generation & Self-Correction

Stage 3: Generation & AST Construction

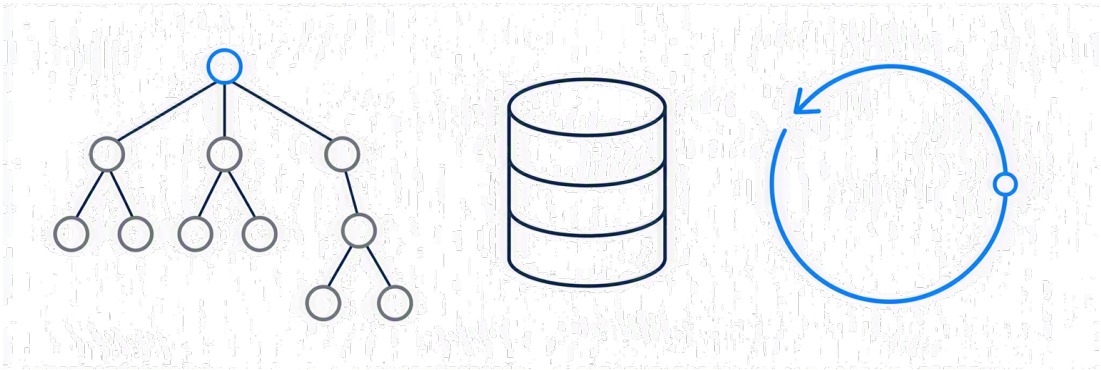
Translating linked components into valid SQL statements:

- **Syntax Rules:** Strict enforcement of SQL clause ordering.
- **Dialect Adaptation:** Formatting specifically for PostgreSQL, MySQL, SQLite, or Snowflake.
- **Subquery Structuring:** Nesting queries for complex conditions.

Stage 4: Agentic Self-Correction

Execution feedback loop prevents query failure:

- **Dry Run Execution:** Runs generated SQL against DB engine.
- **Error Interception:** Catches syntax errors, missing columns, type mismatches.
- **Iterative Repair:** Sends DB error stack trace back to LLM to auto-correct SQL.



Evolution of System Architectures

1. Single-Pass Prompting

Question + Static DDL Schema directly to LLM.

Pros: Fast, zero setup.

Cons: Fails on huge enterprise schemas.

2. Schema RAG

Vector retrieval retrieves only top-N relevant tables.

Pros: Fits large databases in context window.

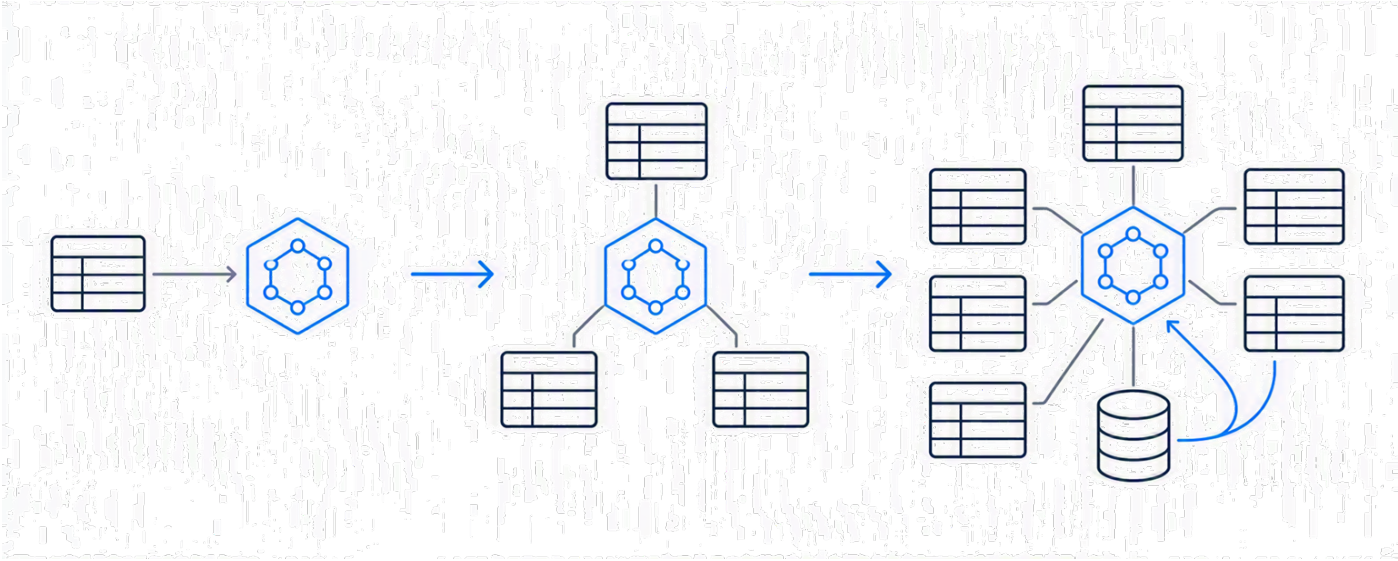
Cons: Can miss indirect FK join paths.

3. Agentic Workflow

LLM Agent with schema lookup, execution, & repair tools.

Pros: Highest accuracy, self-correcting.

Cons: Higher latency and token cost.



Multi-Table Foreign Key Traversal Trace

User Input: ***"Show the names of all students with a GPA above 3.5"***

Step 1: Intent Extraction: Target = `student name`, Condition = `GPA > 3.5`.

Step 2: Schema Linking: Entity `student name` → `students.name`, `GPA` → `students.gpa`. Table = `students`.

Step 3: Clause Assembly: Projection = `SELECT name`, Scope = `FROM students`, Filter = `WHERE gpa > 3.5`.

```
-- Final Generated SQL
SELECT name FROM students WHERE gpa > 3.5;
```

Multi-Table Foreign Key Traversal Trace

User Input: ***"Find teacher names who teach >3 courses taken by students with GPA > 3.0"***

Step 1: Entity Parsing: `teacher names`, `courses`, `students`, `GPA > 3.0`, `course count > 3`.

Step 2: Schema Path Discovery (FK Graph Traversal):
`teachers (teacher_id)` → `courses (teacher_id, course_id)` → `enrollments (course_id, student_id)` → `students (student_id)`.

Step 3: Aggregation & Grouping Logic: Group by `teacher_id`, apply `HAVING COUNT(DISTINCT course_id) > 3`.

```
SELECT t.name FROM teachers t
JOIN courses c ON t.teacher_id = c.teacher_id
JOIN enrollments e ON c.course_id = e.course_id
JOIN students s ON e.student_id = s.student_id
WHERE s.gpa > 3.0 GROUP BY t.teacher_id, t.name
HAVING COUNT(DISTINCT c.course_id) > 3;
```

Agentic Self-Correction Trace

Attempt 1 (LLM Generation Failure):

```
SELECT AVG(salary) FROM teachers;
```

DB Engine Error: ERROR: column "salary" does not exist in table "teachers"

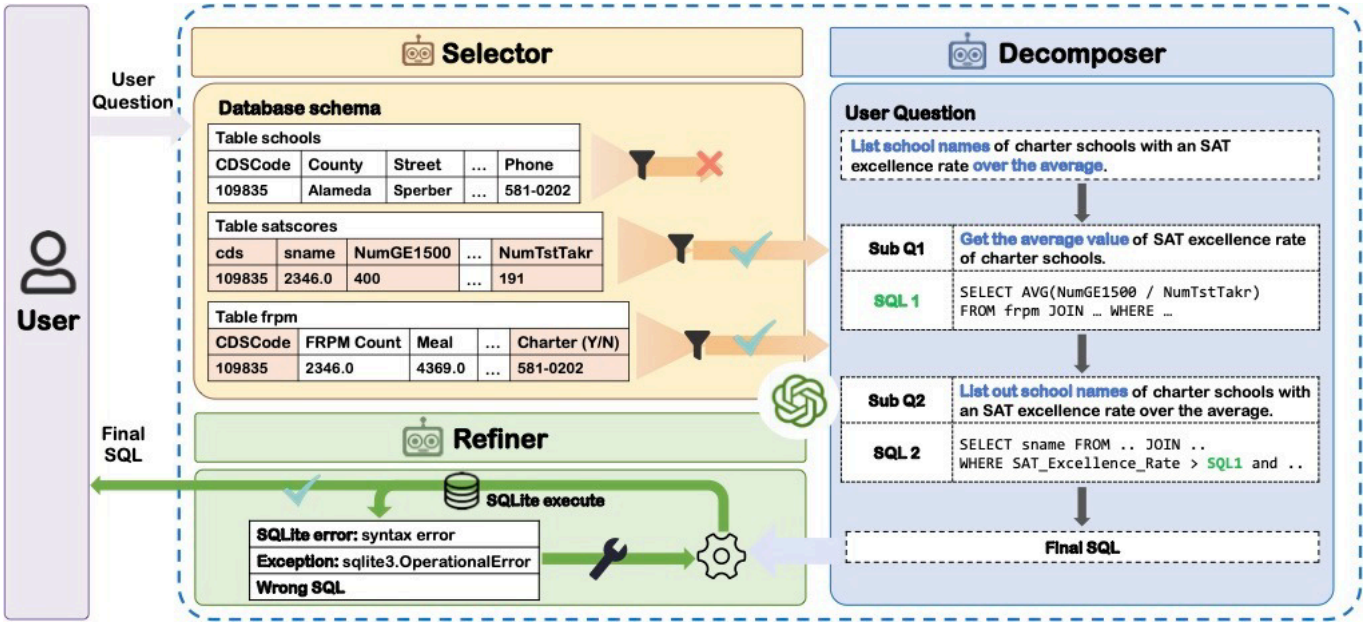
Agent Diagnosis Step:

Agent inspects schema DDL for table teachers. Identifies column monthly_pay as true numeric salary column.

Attempt 2 (Corrected Query Execution):

```
SELECT AVG(monthly_pay) AS avg_teacher_pay FROM teachers;
```

DB Engine Response: SUCCESS (1 row returned: 5420.50)



MAC-SQL:AMulti-Agent Collaborative Framework for Text-to-SQL https://arxiv.org/pdf/2312.11242

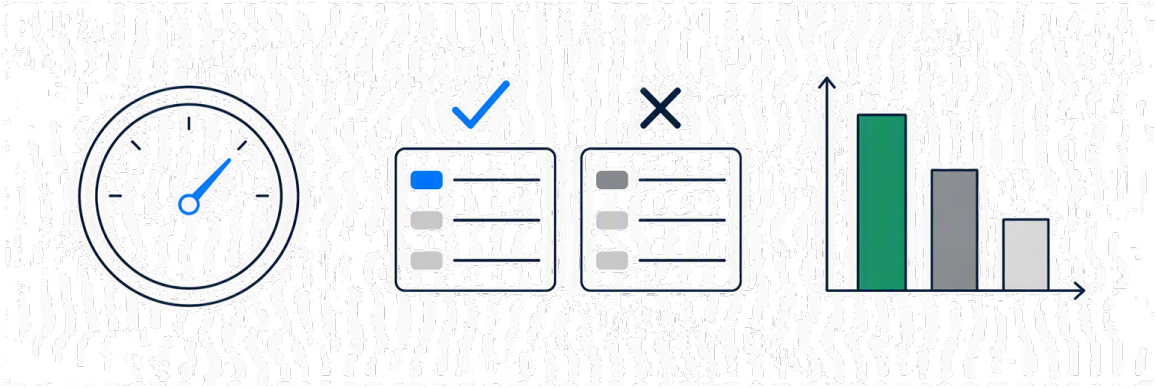
How Text-to-SQL Systems Are Evaluated

Key Evaluation Metrics

- **Exact-Set Match Accuracy:** Compares generated SQL structure with ground-truth query string.
- **Execution Accuracy (EX):** Checks whether executed SQL yields identical database results as ground truth (preferred).
- **Valid SQL Rate:** Percentage of queries that execute without engine syntax errors.

Industry Benchmarks

- **Spider Benchmark:** Standard complex multi-table Text-to-SQL cross-domain dataset. https://yale-lily.github.io/spider
- **BIRD Benchmark:** Real-world enterprise dataset testing noise, dirty data, and dirty schemas. https://bird-bench.github.io/



Summary

- SQL was an important factor in the early acceptance of the relational model
- Even queries that can be expressed in relational algebra can often be expressed more naturally in SQL
- Basic Queries
  - **Select** column **from** relations (tables) **where** qualifications
  - Join operations, Set operations
  - Aggregate functions & Group By, Having and Order By Clauses
- Nested Queries
  - IN, NOT IN, EXISTS, NOT EXISTS, *op* ANY and *op* ALL where *op* ϵ {<. <=, =, <>, >=, >}
  - Re-writing INTERSECT using IN
  - Re-writing EXCEPT using NOT IN
- Null values and Text-to-SQL

Thank you!